

PSpice Simulator

Technical Reference Documentation

MNA-Based SPICE Circuit Simulator with Adjoint Sensitivity Analysis

EECT 7V88 — Digital Twins for Integrated Circuits and Beyond
The University of Texas at Dallas • Spring 2026

Table of Contents

1. Architecture Overview

This document provides a complete technical reference for a Python-based SPICE circuit simulator implementing Modified Nodal Analysis (MNA). Every mathematical equation used in the code is presented alongside its textbook form, with direct references to the implementing source file and line numbers.

The simulator supports DC operating point (.OP), AC small-signal sweep (.AC), transient time-domain (.TRAN) analyses, and adjoint-based sensitivity computation. It handles linear components (R, L, C, V, I), controlled sources (G, E, F, H), nonlinear devices (diodes, MOSFETs, BJTs), and ideal opamps.

1.1 Module Map

Module	Purpose	Key Equations
constants.py	Physical constants	$V_t = kT/q$
parser.py	Netlist parsing	SI suffix conversion
node_index.py	MNA index allocation	Node $\rightarrow$ matrix row mapping
component_stamps.py	Device stamps into Y matrix	All device I-V models
assembleYmatrix.py	Dispatch and matrix assembly	$Y \cdot V = b$ system construction
solver.py	Linear/nonlinear solvers	Newton-Raphson, GMIN stepping
simulations.py	Analysis orchestration	OP, AC, TRAN loops
sources.py	Source evaluation	PULSE, SIN, AC phasors
sensitivity.py	Adjoint sensitivity	$s = \partial \text{output} / \partial \text{parameter}$
adjoint_applications.py	Noise, yield, fault, distortion	Lecture 9 applications
tools.py	Plotting (Bode, transient)	dB conversion, phase
main.py	Entry point, CLI, GUI routing	Orchestration

1.2 The MNA System

Modified Nodal Analysis formulates the circuit as a linear system $Y \cdot V = b$, where Y is the admittance matrix, V is the vector of unknown node voltages and branch currents, and b is the excitation vector. For a circuit with N nodes (excluding ground) and M voltage sources/inductors, the system has dimension $(N+M) \times (N+M)$.

$$Y \cdot V = b$$

The Y matrix is partitioned as:

$$\begin{bmatrix} G & B \\ C & D \end{bmatrix} \begin{bmatrix} v \\ j \end{bmatrix} = \begin{bmatrix} i \\ e \end{bmatrix}$$

where G is the $N \times N$ conductance matrix, B and C are the coupling matrices for voltage sources and inductors, D holds the branch equation coefficients, v is the node voltage vector, j is the branch current vector, i is the independent current source vector, and e is the voltage source value vector.

Note: File: `assembleYmatrix.py` — The `initialize_stamps()` function creates the empty $(N+M) \times (N+M)$ sparse matrix. File: `node_index.py` — `build_node_index()` assigns sequential indices: first all voltage nodes (sorted), then branch currents for V, L, O, E, H components.

2. Physical Constants

File: **constants.py**

2.1 Thermal Voltage

The thermal voltage is a fundamental quantity in semiconductor physics that appears in every PN junction equation:

$$V_T = kT / q$$

where $k = 1.380649 \times 10^{-23}$ J/K is Boltzmann's constant, $T = 298.15$ K is the absolute temperature, and $q = 1.602176634 \times 10^{-19}$ C is the electron charge. At room temperature (25°C):

$$V_T \approx 0.02569 \text{ V} \approx 25.69 \text{ mV}$$

Code Implementation

```
# constants.py
import scipy.constants as c
e = c.e          # 1.602e-19 C
kb = c.k          # 1.381e-23 J/K
T = 298.15        # Kelvin
Vt = (kb * T) / e # ~0.02569 V
```

3. Linear Component Stamps

File: **component_stamps.py**, **assembleYmatrix.py**

Each linear component contributes entries to the Y matrix and/or excitation vector b. The process of adding these entries is called *stamping*. This chapter shows the textbook stamp for each component alongside the code implementation.

3.1 Resistor

A resistor R between nodes i and j obeys Ohm's law: $I = V/R = G \cdot V$, where $G = 1/R$ is the conductance. The MNA stamp adds G to the four corners of the (i,j) submatrix:

$$Y[i,i] += G, \quad Y[i,j] -= G$$

$$Y[j,i] -= G, \quad Y[j,j] += G$$

This enforces KCL at nodes i and j: the current leaving node i through the resistor equals $G \cdot (V_i - V_j)$.

Code (stamp_resistor)

```
g = 1.0 / R
Y[i, i] += g
Y[i, j] -= g
Y[j, i] -= g
Y[j, j] += g
```

Verification

For the voltage divider circuit $V1=10V$, $R1=R2=1k\Omega$: $V(2) = 10 \times R2/(R1+R2) = 10 \times 1000/2000 = \mathbf{5.000V}$. The simulator produces $V(2) = 5.000000$ V.

3.2 Capacitor (AC Domain)

In the frequency domain, a capacitor C has admittance $Y_C = j\omega C$:

$$Y[i,i] += j\omega C, \quad Y[i,j] -= j\omega C$$

$$Y[j,i] -= j\omega C, \quad Y[j,j] += j\omega C$$

Code (stamp_capacitor)

```
g = 1j * w * C
Y[i, i] += g      # same pattern as resistor
```

3.3 Capacitor (Backward Euler)

For transient analysis, the capacitor is discretized using the Backward Euler method. The companion model replaces the capacitor with an equivalent conductance G_{eq} and a history current source I_{eq} :

$$G_{eq} = C / \Delta t$$

$$I_{eq} = G_{eq} \cdot (v1(t-\Delta t) - v2(t-\Delta t))$$

The conductance G_{eq} is stamped identically to a resistor. The history current I_{eq} is added to the RHS vector, representing the charge stored from the previous time step.

Code (stamp_capacitor_be)

```
Y_eq = C / dt
I_eq = Y_eq * (v1_prev - v2_prev)
# Stamp G_eq like a resistor
Y[i1, i1] += Y_eq ; Y[i2, i2] += Y_eq
Y[i1, i2] -= Y_eq ; Y[i2, i1] -= Y_eq
# Stamp history current
sources[i1] += I_eq ; sources[i2] -= I_eq
```

3.4 Inductor

An inductor requires an MNA branch variable for the current I_L . The branch equation relates voltage to current:

AC domain: $V(n1) - V(n2) - j\omega L \cdot I_L = 0$

$$Y[branch, branch] -= j\omega L$$

Backward Euler: The companion model uses $R_{eq} = L/\Delta t$ and history voltage V_{eq} :

$$R_{eq} = L / \Delta t$$

$$V_{eq} = (L / \Delta t) \cdot I_L(t - \Delta t)$$

Code (stamp_inductor_be)

```
R_eq = L / dt
Y[idx, idx] -= R_eq
V_eq = L * i_L_prev / dt
sources[idx] -= V_eq
```

3.5 Independent Voltage Source

A voltage source V between nodes $n+$ and $n-$ adds an MNA branch variable I_V . The topology stamps enforce KCL coupling, and the branch equation enforces $V(n+) - V(n-) = V_s$:

$$Y[i, branch] += 1, Y[branch, i] += 1 \quad (\text{for } n+)$$

$$Y[j, branch] -= 1, Y[branch, j] -= 1 \quad (\text{for } n-)$$

$$b[branch] = V_s$$

Note: The topology stamps (1/-1) are applied once by `stamp_mna_connection()`. The value V_s is stamped separately by `stamp_independent_voltage()`, allowing transient re-stamping without duplicating the topology.

3.6 Independent Current Source

A current source I from $n+$ to $n-$ stamps directly into the RHS vector:

$$b[i] -= I \quad (\text{current leaves } n+)$$

$$b[j] += I \quad (\text{current enters } n-)$$

3.7 Controlled Sources (G, E, F, H)

The four SPICE controlled sources implement linear dependencies between voltages and currents:

Type	Name	Equation	MNA Branch?
G (VCCS)	Voltage-Controlled Current Source	$I_{\text{out}} = g \cdot (V_{\text{c}+} - V_{\text{c}-})$	No
E (VCVS)	Voltage-Controlled Voltage Source	$V_{\text{out}} = A \cdot (V_{\text{c}+} - V_{\text{c}-})$	Yes
F (CCCS)	Current-Controlled Current Source	$I_{\text{out}} = \alpha \cdot I_{\text{ctrl}}$	No
H (CCVS)	Current-Controlled Voltage Source	$V_{\text{out}} = r \cdot I_{\text{ctrl}}$	Yes

VCCS (G) Stamp

The transconductance g multiplies the sensing voltage and injects current at the output nodes:

$$Y[\text{out}+, \text{sense}+] += g, \quad Y[\text{out}+, \text{sense}-] -= g$$

$$Y[\text{out}-, \text{sense}+] -= g, \quad Y[\text{out}-, \text{sense}-] += g$$

VCVS (E) Stamp

Uses an MNA branch equation: $V(n+) - V(n-) - A \cdot (V(nc+) - V(nc-)) = 0$. The topology 1/-1 stamps come from `stamp_mna_connection`, and the gain terms are:

$$Y[\text{branch}, nc+] -= A, \quad Y[\text{branch}, nc-] += A$$

CCCS (F) Stamp

References the branch current of a controlling voltage source V_{ctrl} . No MNA branch of its own:

$$Y[\text{out}+, I_{\text{ctrl_col}}] -= \alpha, \quad Y[\text{out}-, I_{\text{ctrl_col}}] += \alpha$$

Note: The sign follows from SPICE convention: positive I_{ctrl} means current entering the controlling source's positive terminal. The dependent source copies this current (scaled by α) into the output node.

CCVS (H) Stamp

Branch equation: $V(n+) - V(n-) - r \cdot I_{\text{ctrl}} = 0$. Topology from `stamp_mna_connection`, then:

$$Y[\text{branch}, I_{\text{ctrl_col}}] -= r$$

4. Nonlinear Device Models

File: **component_stamps.py**

Nonlinear devices are handled via Newton-Raphson linearization. At each iteration, the nonlinear I-V characteristic is linearized around the current guess V_k , producing a companion model (equivalent conductance + current source) that is stamped into the Y matrix.

4.1 PN Junction Voltage Limiting (pnjlim)

Before evaluating exponential junction equations, the voltage guess is *limited* to prevent numerical overflow. This is the standard SPICE technique:

$$\text{If } V_{\text{new}} > V_{\text{crit}} \text{ and } |V_{\text{new}} - V_{\text{old}}| > 2V_T:$$

$$V_{\text{limited}} = V_{\text{old}} + 2V_T \cdot \ln(V_{\text{new}} / V_{\text{old}})$$

This logarithmic limiting ensures the junction voltage changes by at most a few thermal voltages per iteration, preventing the exponential from overflowing.

Code (pnjlim)

```
if v_new > critical_v and abs(v_new - v_old) > (2 * Vt):  
    v_limit = v_old + 2 * Vt * ln(max(v_new / v_old, 1e-30))
```

4.2 Diode (Shockley Model)

The diode forward current follows the Shockley equation:

$$I_D = I_S \cdot (\exp(V_D / NV_T) - 1)$$

where I_S is the saturation current, N is the emission coefficient, and V_T is the thermal voltage. The Newton companion model linearizes this around the operating point $V_{D,k}$:

$$g_d = (I_S / NV_T) \cdot \exp(V_{D,k} / NV_T) \quad (\text{conductance})$$

$$I_{eq} = I_D(V_{D,k}) - g_d \cdot V_{D,k} \quad (\text{equivalent current})$$

The companion model stamps g_d like a resistor and I_{eq} into the RHS. Optional parameters include series resistance R_S (solved with a fixed-point iteration) and reverse breakdown (BV, IBV, NBV).

Code (stamp_diode)

```
Vte = max(N * Vt, 1e-12)  
arg_f = clip(vj / Vte, -50.0, 50.0)  
ef = exp(arg_f)  
Ifwd = Is * (ef - 1.0)  
gfwd = (Is / Vte) * ef  
  
# Newton companion stamps:  
ieq = Id - gd * vd_lim  
Y[idx1, idx1] += gd ; Y[idx2, idx2] += gd  
Y[idx1, idx2] -= gd ; Y[idx2, idx1] -= gd
```

```
sources[idx1] -= ieq ; sources[idx2] += ieq
```

Verification

For $V_1=5V$, $R_1=1k\Omega$, D_1 ($I_S=1e-14$) to ground: solving KCL $(5-V_d)/1000 = I_S \cdot (\exp(V_d/V_t)-1)$ gives $V_d \approx 0.6883V$. The simulator produces $V(2) = \mathbf{0.688298V}$, matching to 5 significant figures.

4.3 MOSFET (Level 1)

The Level 1 MOSFET model uses the Shichman-Hodges equations:

NMOS Drain Current

$$\text{Cutoff } (V_{GS} < V_{TH}): I_D = 0$$

$$\text{Triode } (V_{DS} < V_{GS} - V_{TH}): I_D = \beta_n \cdot [(V_{GS} - V_{TH}) \cdot V_{DS} - \frac{1}{2} V_{DS}^2]$$

$$\text{Saturation } (V_{DS} \geq V_{GS} - V_{TH}): I_D = \frac{1}{2} \beta_n \cdot (V_{GS} - V_{TH})^2$$

where $\beta_n = (W/L) \cdot K_P$ is the gain factor.

Linearization Conductances

$$g_m = \partial I_D / \partial V_{GS} = \beta_n \cdot V_{DS} \text{ (triode) or } \beta_n \cdot (V_{GS} - V_{TH}) \text{ (saturation)}$$

$$g_{ds} = \partial I_D / \partial V_{DS} = \beta_n \cdot (V_{GS} - V_{TH} - V_{DS}) \text{ (triode) or } \epsilon \text{ (saturation)}$$

PMOS

PMOS uses complementary voltages $V_{SG} = V_S - V_G$ and $V_{SD} = V_S - V_D$ with $|V_{TH}|$ as threshold. The Y matrix stamp pattern is identical to NMOS; only the equivalent current source direction is reversed.

Code (stamp_mosfet)

```
# NMOS saturation
Id = 0.5 * Bn * (vov ** 2)
gm = Bn * vov
gds = 1e-12 # small for saturation

# Stamp: collector KCL
Y[idx_d, idx_g] += gm ; Y[idx_d, idx_s] -= gm
Y[idx_d, idx_d] += gds ; Y[idx_d, idx_s] -= gds
# ... (symmetric for source node)
Ieq = Id - (gm * vgs_k) - (gds * vds_k)
sources[idx_d] -= Ieq ; sources[idx_s] += Ieq
```

Verification

CMOS inverter ($V_{DD}=5V$, $K_{P_n}=100\mu A/V^2$, $K_{P_p}=50\mu A/V^2$, $V_{TH}=0.7V$): When $V_{IN}=0V$, PMOS is ON ($V_{SG}=5V$), NMOS is OFF $\rightarrow V_{OUT}=5V$. When $V_{IN}=5V$, NMOS is ON, PMOS is OFF $\rightarrow V_{OUT}=0V$. Simulator confirms: $V(out, t=0)=5.0000V$, $V(out, t=5\mu s)=0.0000V$.

4.4 BJT (Ebers-Moll Transport Model)

The BJT uses the Ebers-Moll transport model with two PN junctions:

Transport Currents

$$I_F = I_S \cdot (\exp(V_{BE} / N_F V_T) - 1) \quad (\text{forward})$$

$$I_R = I_S \cdot (\exp(V_{BC} / N_R V_T) - 1) \quad (\text{reverse})$$

Terminal Currents

$$I_C = I_F - I_R \quad (\text{collector})$$

$$I_B = I_F / \beta_F + I_R / \beta_R \quad (\text{base})$$

$$I_E = -(I_C + I_B) \quad (\text{emitter, KCL})$$

Early Effect

The forward Early voltage V_{AF} introduces output resistance:

$$I_C = I_F \cdot (1 + V_{BC}/V_{AF}) - I_R$$

Linearization (Newton Companion)

$$g_m = \partial I_F / \partial V_{BE} \cdot (1 + V_{BC}/V_{AF}) \quad (\text{transconductance})$$

$$g_o = \partial I_R / \partial V_{BC} + I_F / V_{AF} \quad (\text{output conductance})$$

$$g_\pi = g_m / \beta_F \quad (\text{base input conductance})$$

$$g_\mu = (\partial I_R / \partial V_{BC}) / \beta_R \quad (\text{base-collector feedback})$$

MNA Stamp Pattern (NPN)

The linearized BJT stamps four conductances into the Y matrix. The stamp pattern at collector (c), base (b), and emitter (e) nodes:

Y entry	Value	Physical meaning
Y[c,b]	$(g_m - g_o)$	Collector current depends on V_B
Y[c,e]	$-g_m$	Collector current depends on V_E
Y[c,c]	g_o	Output resistance (Early effect)
Y[b,b]	$g_\pi + g_\mu$	Base input admittance
Y[b,e]	$-g_\pi$	B-E junction conductance
Y[b,c]	$-g_\mu$	B-C feedback
Y[e,b]	$-(g_m - g_o + g_\pi + g_\mu)$	Emitter KCL (sum)
Y[e,e]	$g_m + g_\pi$	Emitter self-conductance
Y[e,c]	$g_o + g_\mu$	Emitter-collector coupling

PNP

For PNP: all terminal voltages are negated internally (V_{EB} , V_{CB} replace V_{BE} , V_{BC}), and the equivalent current sources are sign-flipped. The Y conductance stamps remain identical to NPN.

Verification

Common-emitter amplifier ($V_{CC}=10V$, $R_B=100k\Omega$, $R_C=1k\Omega$, $V_{BB}=2V$, $\beta_F=100$, $I_S=1e-14$): $I_b = (2-V_{be})/100k = 13.4\mu A$, $I_c = \beta \cdot I_b = 1.34mA$, $V_c = 10 - 1.34 \times 1k = 8.66V$. Simulator produces: $V_{be}=0.6583V$, $I_c=1.342mA$, $\beta=100.0$, $V_c=8.658V$.

5. Solver Architecture

File: **solver.py**

5.1 Linear Solve

For linear circuits, the system $Y \cdot V = b$ is solved directly using sparse LU factorization (SuperLU via SciPy):

$$P \cdot L \cdot U = Y \rightarrow V = U^{-1} L^{-1} P \cdot b$$

The LU factorization is retained for adjoint sensitivity computation.

5.2 Newton-Raphson for Nonlinear Circuits

Nonlinear circuits require iterative solution. At each iteration k :

1. *Linearize all nonlinear devices around V_k*
2. *Assemble Y_k and b_k (including companion models)*
3. *Solve $Y_k \cdot V_{k+1} = b_k$*
4. *Check convergence: $\max|V_{k+1} - V_k| < \text{tolerance}$*

Convergence tolerance: 1×10^{-6} V. Maximum iterations: 200.

5.3 Convergence Strategies

The solver uses a three-stage strategy to handle difficult circuits:

Stage	Method	Best For
1	Direct NR with small GMIN (10^{-12})	Well-conditioned circuits
2	GMIN stepping: $0.1 \rightarrow 10^{-12}$	Floating nodes (CMOS, diodes)
3	Source ramping: 10% $\rightarrow$ 100%	BJTs (need full supply)

GMIN is a small conductance added to every voltage-node diagonal, preventing singular matrices when nodes are floating. It is the standard SPICE technique.

Source ramping scales the RHS vector from 10% to 100% in steps, solving at each level. This works for BJTs where GMIN stepping fails because large GMIN drowns out the junction conductances.

Initial guess: The solver first tries a linear-subcircuit solution (ignoring nonlinear devices) to establish supply voltages. If GMIN stepping is needed, it starts from zeros to avoid high initial junction voltages.

6. Analysis Modes

File: **simulations.py**

6.1 DC Operating Point (.OP)

Solves $Y \cdot V = b$ at DC ($\omega=0$). Capacitors are open circuits (removed), inductors are short circuits ($V=0$ across them). Nonlinear devices use Newton-Raphson.

6.2 AC Small-Signal Sweep (.AC)

Linearizes nonlinear devices at the DC bias point, then sweeps frequency. At each frequency f :

$$\omega = 2\pi f$$

$$Y_{AC}(\omega) = Y_{DC} + j\omega C \text{ (capacitors)} + 1/(j\omega L) \text{ (inductors)}$$

$$V_{AC}(\omega) = Y_{AC}^{-1}(\omega) \cdot b_{AC}$$

AC sources are represented as complex phasors: $V_{AC} = |V| \cdot \exp(j\phi)$.

6.3 Transient Analysis (.TRAN)

Time-marches from $t=0$ to t_{stop} with step Δt using Backward Euler discretization. At each time step:

1. Evaluate time-dependent sources at time t
2. Stamp BE companion models using $v(t-\Delta t)$
3. Solve (linear or Newton-Raphson)
4. Store solution, advance $t += \Delta t$

Source Functions

Function	Equation
PULSE	V1 during delay, ramp V1→V2 over TR, hold V2 for PW, ramp back over TF, repeat with period PER
SIN	$V = V_{OFF} + V_{AMP} \cdot \sin(2\pi f \cdot t + \phi)$
COS	$V = V_{OFF} + V_{AMP} \cdot \cos(2\pi f \cdot t + \phi)$

7. Adjoint Sensitivity Analysis

Files: **sensitivity.py**, **solver.py** (solve_adjoint)

The adjoint method computes the sensitivity of a scalar output (e.g., $V(\text{node})$) with respect to *all* circuit parameters simultaneously, using only two solves: one forward, one adjoint.

7.1 Mathematical Foundation

Given the circuit equation $Y \cdot V = b$, the sensitivity of an output $V_{\text{out}} = e^T \cdot V$ to parameter p is:

$$s = \partial V_{\text{out}} / \partial p = -\Psi^T \cdot (\partial Y / \partial p) \cdot V$$

where Ψ is the **adjoint vector**, solved from:

$$Y^T \cdot \Psi = e \quad (e \text{ is a unit vector at the output node})$$

This requires the **transpose** of the same LU factorization used for the forward solve.

Code (solve_adjoint)

```
def solve_adjoint(lu, target, node_map):
    d = np.zeros(len(node_map))
    d[node_map[target]] = 1.0
    return lu.solve(d, trans="T") # transpose solve
```

7.2 Per-Component Sensitivity Formulas

The derivative $\partial Y / \partial p$ depends on the component type. For each component, the sensitivity formula combines the forward solution V , the adjoint vector Ψ , and the parameter derivative of the device model.

Component	Parameter	Sensitivity Formula
Resistor R	R	$s = -(1/R^2) \cdot (V_i - V_j) \cdot (\Psi_i - \Psi_j)$
Capacitor C	C (AC)	$s = -j\omega \cdot (V_i - V_j) \cdot (\Psi_i - \Psi_j)$
Capacitor C	C (transient)	$s = -(1/\Delta t) \cdot (V_i - V_j) \cdot (\Psi_i - \Psi_j)$
Inductor L	L (AC)	$s = j\omega \cdot I_L \cdot \Psi_L$
Voltage src V	V	$s = \Psi_{\text{branch}}$
Current src I	I	$s = -(\Psi_i - \Psi_j)$
VCCS G	g	$s = -(V_{\text{sense}}) \cdot (\Psi_i - \Psi_j)$
Diode D	IS, N, RS, ...	$s = -(\Psi_i - \Psi_j) \cdot \partial I_D / \partial p \text{ (finite diff)}$
MOSFET M	VTO, KP, ...	$s = -(\Psi_d - \Psi_s) \cdot \partial I_D / \partial p \text{ (finite diff)}$
BJT Q	IS, BF, NF, ...	$s = -(\Psi_c - \Psi_e) \cdot \partial I_C / \partial p \text{ (finite diff)}$

Note: For nonlinear devices, the parameter derivatives are computed using central finite differences: $\partial I / \partial p \approx [I(p+\Delta p) - I(p-\Delta p)] / (2\Delta p)$, with $\Delta p = \max(10^{-6}|p|, 10^{-12})$.

7.3 Output Standard Deviation

Given sensitivities s_k and parameter tolerances σ_k , the output standard deviation (assuming independent, Gaussian parameters) is:

$$\sigma_{out} = \sqrt{\sum_k (s_k \cdot \sigma_k)^2}$$

Code (estimate_std_dev)

```
sigma_p = base * percent_sigma
variance += abs(sens * sigma_p) ** 2
return sqrt(variance)
```


8. Adjoint Applications (Lecture 9)

File: **adjoint_applications.py**, integrated via **main.py**

Based on Prof. Ron Rohrer's Lecture 9 (EECT 7V88, Spring 2026), six applications of adjoint sensitivity are implemented. The key insight: a single forward solve gives input→device transfers, and a single adjoint solve gives device→output transfers. Together, they provide output sensitivity to all parameters.

8.1 Noise Estimation

Each device contributes a noise current/voltage source with spectral density $S_k(f)$. The adjoint vector Ψ transfers each device's noise to the output:

$$S_{out}(f) = \sum_k |\Psi_k|^2 \cdot S_k(f) \quad [V^2/Hz]$$

Device Noise Models

Device	Noise Type	Spectral Density $S(f)$
Resistor R	Thermal (Johnson-Nyquist)	$4kT/R$ $[A^2/Hz]$
Diode D	Shot noise	$2qI_D$ $[A^2/Hz]$
MOSFET M	Channel thermal	$4kT\gamma g_m$ $[A^2/Hz]$ ($\gamma=2/3$)
BJT Q (Ic)	Collector shot	$2qI_C$ $[A^2/Hz]$
BJT Q (Ib)	Base shot	$2qI_B$ $[A^2/Hz]$

Verification

Voltage divider ($R1=R2=1k\Omega$): Both resistors contribute equally → 50%/50% split. Simulator confirms: $R1=50.0\%$, $R2=50.0\%$.

Diode circuit ($R1=1k\Omega$, D1 at 4.3mA): Diode shot noise dominates (dynamic resistance $\sim 6\Omega \ll R1$). Simulator: $D1=98.8\%$, $R1=1.2\%$.

BJT CE amplifier: Q1 dominates at 70.9% (collector shot noise + base shot), with bias resistors contributing 22.6% and 4.5%.

8.2 Yield Estimation

Propagates parameter tolerances through sensitivities to predict manufacturing yield:

$$\sigma_{out}^2 = \sum_k (s_k \cdot \sigma_k)^2$$

$$Cpk = \min(V_{out} - Spec_{min}, Spec_{max} - V_{out}) / (3\sigma_{out})$$

$$Yield \approx \Phi(\text{margin}_{high}) + \Phi(\text{margin}_{low}) - 1$$

where Φ is the standard normal CDF, and margins are in units of σ .

8.3 Fault Dictionary

For each component, predicts the output shift under various fault magnitudes using first-order sensitivity:

$$\Delta V_{out} \approx s_k \cdot \Delta p_k$$

Default fault conditions: $\pm 10\%$ and $\pm 50\%$. The `diagnose_fault()` function performs reverse lookup: given a measured output deviation, it ranks which component faults best explain it.

8.4 Distortion Analysis

Uses Taylor expansion of nonlinear device I-V to estimate harmonic generation, then adjoint at each harmonic frequency to find output contributions:

$$HD2 = |V(2f_0)| / |V(f_0)|, \quad HD3 = |V(3f_0)| / |V(f_0)|$$
$$THD = \sqrt{HD2^2 + HD3^2}$$

8.5 Design Centering

Uses sensitivity gradients to shift parameters toward the center of the specification window, maximizing manufacturing margin:

$$\Delta p_k = \eta \cdot (V_{center} - V_{nominal}) \cdot s_k \cdot \sigma_k / \Sigma(s_j \cdot \sigma_j)^2$$

where η is a learning rate. This is a single gradient descent step in the normalized parameter space.

8.6 Radiation Sensitivity (Planned)

Requires time-domain adjoint (backwards-in-time integration). The forward transient infrastructure exists; the reverse sweep is not yet implemented. This is noted as *difficult but doable* in the lecture.

9. Netlist Parser

File: **parser.py**

The parser reads SPICE-format netlists and produces a components dictionary and an analyses dictionary. It supports line continuations (+), comments (*), and .END termination.

9.1 Supported Components

Letter	Component	Format
R, L, C	Passive	Name N1 N2 Value
V, I	Source	Name N+ N- [DC val] [AC mag [phase]] [TRAN_FUNC]
D	Diode	Name N+ N- Model_or_Value
M	MOSFET	Name ND NG NS NB Model [W=... L=...]
Q	BJT	Name NC NB NE [NS] Model
G	VCCS	Name N+ N- NC+ NC- Gain
E	VCVS	Name N+ N- NC+ NC- Gain
F	CCCS	Name N+ N- Vcontrol Gain
H	CCVS	Name N+ N- Vcontrol Gain
O	Opamp	Name Nout Nref N+ N- [Gain]

9.2 SI Suffix Parsing

Suffix	Multiplier	Example
T	10^{12}	1T = 1e12
G	10^9	2.2G = 2.2e9
MEG	10^6	10MEG = 1e7
K	10^3	4.7K = 4700
M	10^{-3}	100M = 0.1
U	10^{-6}	1U = 1e-6
N	10^{-9}	100N = 1e-7
P	10^{-12}	1P = 1e-12
F	10^{-15}	10F = 1e-14

10. Complete Worked Example

This section traces a complete simulation from netlist to results, showing every equation evaluation.

10.1 Circuit: BJT Common-Emitter Amplifier

```
VCC 1 0 10
RC 1 2 1000
RB 3 4 100000
VBB 3 0 2
Q1 2 4 0 QMOD
.MODEL QMOD NPN IS=1e-14 BF=100 NF=1
.OP
```

10.2 Step 1: Parse & Build Node Map

Parser identifies 5 components. Node map: {1:0, 2:1, 3:2, 4:3, VCC:4, VBB:5}. Total unknowns: 6 (4 voltages + 2 branch currents).

10.3 Step 2: Stamp Linear Components

RC (1k Ω , nodes 1 $\rightarrow$ 2): $G=0.001\text{S}$ stamped at $Y[0,0]$, $Y[1,1]$, $Y[0,1]$, $Y[1,0]$. RB (100k Ω , nodes 3 $\rightarrow$ 4): $G=10^{-5}\text{S}$. VCC: topology at rows/cols [0,4]. VBB: topology at rows/cols [2,5]. Source values: $b[4]=10$, $b[5]=2$.

10.4 Step 3: Newton-Raphson (BJT)

Since Q1 is detected as nonlinear, NR is invoked. The linear initial guess gives $V=[10, 9.99, 2.0, 1.82, \dots]$. The solver attempts direct NR with $\text{GMIN}=10^{-12}$, which converges in ~ 15 iterations.

At convergence: $V_B=0.6583\text{V}$, $V_C=8.658\text{V}$. The BJT operates in forward active:
 $V_{BE}=0.658\text{V}$, $I_C = I_S \cdot \exp(V_{BE}/V_T) = 10^{-14} \cdot \exp(0.658/0.02569) = 1.34\text{mA}$. $V_C = 10 - 1.34 \times 10^{-3} \times 1000 = 8.66\text{V}$ ✓.

10.5 Step 4: Adjoint Sensitivity

For output node 2 (collector): solve $Y^T \cdot \Psi = e_2$. Then for each parameter:

$s(\text{RC}) = -(1/\text{RC}^2) \cdot (V_1 - V_2) \cdot (\Psi_1 - \Psi_2)$ computes how RC tolerance affects V_C .

$s(\text{Q1:BF}) = -(\Psi_C - \Psi_E) \cdot \partial I_C / \partial \beta_F$ uses finite differences on the Ebers-Moll model.

10.6 Step 5: Noise Analysis

Using the adjoint vector Ψ from Step 4, noise contributions are computed. Q1 collector shot noise: $S = 2q \cdot I_C = 2 \times 1.6 \times 10^{-19} \times 1.34 \times 10^{-3} = 4.29 \times 10^{-22} \text{ A}^2/\text{Hz}$. Transferred to output: $|\Psi_{CE}|^2 \cdot S$.

Appendix A: Model Parameters

A.1 Diode (.MODEL name D)

Parameter	Default	Description
IS	1e-14	Saturation current (A)
N	1.0	Emission coefficient
RS	0.0	Series resistance (Ω)
BV	0.0	Reverse breakdown voltage (V)
IBV	1e-3	Current at breakdown (A)
NBV	1.0	Breakdown emission coefficient

A.2 MOSFET (.MODEL name NMOS/PMOS)

Parameter	Default	Description
VTO (VT0, VTH)	—	Threshold voltage (V)
KP	—	Transconductance parameter (A/V^2)
LAMBDA	0.0	Channel-length modulation (parsed, not yet used in stamp)
W (instance)	1.0	Gate width
L (instance)	1.0	Gate length

A.3 BJT (.MODEL name NPN/PNP)

Parameter	Default	Description
IS	1e-14	Transport saturation current (A)
BF	100	Forward current gain (β_F)
BR	1.0	Reverse current gain (β_R)
NF	1.0	Forward emission coefficient
NR	1.0	Reverse emission coefficient
VAF	0.0	Forward Early voltage (V)
VAR	0.0	Reverse Early voltage (V)